ESPRIT · 4NIDS6 · MODULE DEVOPS

Sécuriser et livrer une API Spring Boot : une chaîne DevSecOps de bout en bout

Rapport de synthèse en français

Oussama Koudhai

Application de gestion des étudiants

12 novembre 2026

Exécution de référence : n° 43082725820

> Ce document est la synthèse française du rapport complet en anglais (*documentation/FINAL-REPORT.pdf*) consacré au même projet. Il présente les résultats par risque traité ; le rapport complet détaille chaque étape et contient les rapports d'outils originaux.

Sommaire

1. L'essentiel en une page
2. Contexte et contraintes
3. Organisation de la chaîne
4. Les risques traités
5. Une exécution réelle, minute par minute
6. Preuves sélectionnées
7. Incidents rencontrés et corrections
8. Limites et plan d'action
9. Conclusion

> Toutes les valeurs proviennent de l'exécution GitHub Actions n° 43082725820 sur la branche master, sauf la partie historique Azure et les incidents datés.

1. L'essentiel en une page

À chaque push sur *master*, une chaîne automatisée contrôle le code et l'infrastructure, construit et analyse l'application, la déploie dans un environnement de test éphémère, vérifie son exposition réseau et sa supervision, puis envoie un courriel de statut.

10 min 15 s du push au courriel de statut	**16 / 17** contrôles réussis, 1 point d'attention	**0** vulnérabilité élevée ou critique dans l'image	**108** paquets Java inventoriés (SBOM)
0 échec Checkov (27 et 43 contrôles)	**41,5 %** de couverture de code (6 tests)	**11** problèmes SonarQube encore ouverts	**13** mises à jour Dependabot en attente

Ce qui fonctionne. Seize contrôles automatisés réussissent et le dix-septième, Dependabot, signale des points d'attention sans bloquer ; chaque outil produit son propre rapport, conservé 30 jours avec l'exécution ; un même artefact testé est analysé, stocké, déployé par empreinte et vérifié.

Ce qui reste à faire. Six problèmes de sécurité de sévérité élevée signalés par SonarQube restent ouverts, les alertes de sécurité Dependabot sont désactivées et l'API n'a pas d'authentification : elle ne doit contenir que des données fictives.

2. Contexte et contraintes

Le point de départ est une API REST Spring Boot de gestion des étudiants, des départements et des inscriptions, initialement livrée avec Jenkins et Minikube. Le projet l'a étendue en chaîne DevSecOps : contrôles de sécurité avant compilation, qualité, artefacts, images analysées, déploiement, acceptation réseau, supervision et maintenance des dépendances.

Deux environnements successifs

Période	Environnement	Statut
Jusqu'au 5 novembre 2026	Deux machines virtuelles Azure (application, puis SonarQube et Nexus), provisionnées par Terraform	Livraison complète réussie (exécution n° 42108525318)
Depuis le 11 novembre 2026	Runner GitHub gratuit (4 processeurs, 16 Go) : tous les outils sont démarrés pour l'exécution puis détruits	Exécutions réussies sur master

Le changement a été imposé par la fin de l'abonnement Azure for Students : la machine des outils est devenue injoignable. Plutôt que de payer deux machines virtuelles en continu, la chaîne recrée à chaque exécution SonarQube, Nexus, un registre d'images et l'application, avec les mêmes fichiers Docker Compose que sur Azure.

> **Conséquence à garder en tête.** Il n'existe plus d'adresse permanente à ouvrir : les preuves sont les rapports, les captures et le courriel produits par chaque exécution. La « production » est un contrôle final sur la même pile éphémère, pas un environnement séparé.

3. Organisation de la chaîne

La chaîne suit les phases du cours. Chaque phase ne démarre que si la précédente a réussi, à l'exception de la maintenance des dépendances, du rapport combiné et du courriel, qui s'exécutent dans tous les cas pour rendre compte.



Rôle de chaque outil

Phase	Outils	Ce qu'ils garantissent
Développement	pre-commit, Gitleaks, Semgrep, Trivy, Checkov, modèle de menaces STRIDE	Rien n'est compilé si un secret, une faille simple, une dépendance vulnérable ou une erreur d'infrastructure est détecté.
Acceptation	Maven, JUnit, JaCoCo, SonarQube, Nexus, Docker, Trivy, registre OCI, Docker Compose, Nmap	Le même JAR testé devient une image analysée, déployée par empreinte immuable, dont l'exposition réseau est vérifiée.
Production	Inspection Docker, requête API	L'image en service est bien celle qui a été analysée, et l'API lit la base de données.
Exploitation	Prometheus, Node Exporter, Grafana	Les métriques de l'application et de l'hôte sont collectées et visibles.
Maintenance	Dependabot, courriel SMTP	Les mises à jour en attente et le résultat de chaque exécution sont signalés.

4. Les risques traités

Plutôt qu'une liste d'outils, cette matrice part des risques : pour chacun, le contrôle qui le réduit, l'outil, et ce que l'exécution de référence a mesuré.

Risque	Contrôle	Outil	Mesure	Effet
Secret publié dans le dépôt	Analyse de tout l'historique Git et du contenu indexé avant commit	Gitleaks, pre-commit	Aucune fuite	Bloquant
Code applicatif vulnérable	Analyse statique à chaque push, porte de qualité avant livraison	Semgrep, SonarQube	19 fichiers, 0 alerte Semgrep ; porte SonarQube franchie, 11 problèmes ouverts	Bloquant
Bibliothèque vulnérable	Analyse du POM avant compilation, inventaire CycloneDX	Trivy	108 paquets, 0 vulnérabilité élevée ou critique	Bloquant
Infrastructure mal configurée	Politiques Terraform, règle maison contre SSH ouvert à Internet	Checkov	27 et 43 contrôles réussis sur deux configurations, 0 échec	Bloquant
Image conteneur vulnérable	Analyse de l'image finale construite à partir du JAR testé	Trivy	0 vulnérabilité élevée ou critique	Bloquant
Artefact substitué	Somme SHA-1 du JAR stocké comparée au JAR testé ; déploiement par empreinte immuable	Nexus, registre OCI	SHA-1 identiques (91d61c45…)	Bloquant
Port exposé par erreur	Balayage TCP de 1 041 ports depuis l'interface réseau et la boucle locale	Nmap	Seuls le SSH du runner et les 4 ports attendus sont ouverts	Bloquant
Régression fonctionnelle	Tests unitaires et HTTP, puis requête réelle sur l'API déployée	Maven, JUnit, JaCoCo	6 tests réussis ; l'API renvoie des données de la base	Bloquant
Panne non détectée	Collecte de métriques et tableaux de bord pendant l'exécution	Prometheus, Grafana	2 cibles sur 2 actives, 30 requêtes sur 30 réussies	Bloquant
Dépendances qui vieillissent	Propositions hebdomadaires de mise à jour pour 5 écosystèmes	Dependabot	13 propositions ouvertes, alertes de sécurité désactivées	Signalé

« Bloquant » : un échec arrête la chaîne avant la livraison. « Signalé » : le résultat est mis en évidence dans le rapport sans bloquer.

5. Une exécution réelle, minute par minute

Exécution n° 43082725820 du 11 novembre 2026 (heures UTC). Durée totale : **10 minutes 15 secondes**.

> **18:58:01** **Push sur master**
> Déclenchement automatique de la chaîne.
>
> **18:58:04** **Contrôles de développement**
> Hooks, Semgrep, Gitleaks, Trivy (POM) et Checkov en parallèle, terminés à 18:59:20, avant toute compilation.
>
> **18:59:22** **Préparation du runner**
> Libération d'espace disque, puis démarrage de SonarQube et Nexus à 19:01:03.
>
> **19:01:33** **Compilation et tests**
> Maven, 6 tests JUnit et mesure de couverture JaCoCo.
>
> **19:03:19** **Qualité puis artefact**
> Analyse SonarQube et porte de qualité, puis publication et vérification dans Nexus à 19:04:00.
>
> **19:04:13** **Image et registre**
> Construction de l'image, analyse Trivy, puis publication dans le registre à 19:04:47.
>
> **19:04:57** **Acceptation**
> Démarrage du laboratoire Docker Compose, puis balayage Nmap de 19:05:39 à 19:06:21.
>
> **19:06:21** **Production**
> Vérification de l'empreinte de l'image en service et d'une requête réelle à l'API.
>
> **19:06:22** **Exploitation**
> Métriques Prometheus et tableaux Grafana, jusqu'à 19:07:22.
>
> **19:07:30** **Maintenance des dépendances**
> Relevé des propositions Dependabot et de l'état des alertes.
>
> **19:08:09** **Courriel de statut**
> Résultat de chaque étape et lien vers l'exécution, envoyés à 19:08:16.

> Le job principal dure 8 minutes, dont 1 min 41 s de préparation du runner. SonarQube et Nexus y sont démarrés pendant la compilation, puis arrêtés après leurs étapes pour libérer la mémoire avant le déploiement de l'application.

6. Preuves sélectionnées

Captures réalisées automatiquement par l'exécution elle-même (navigateur sans interface, adresses locales du runner), sauf le courriel. Pour Nexus, la preuve retenue n'est pas une capture mais la comparaison des sommes SHA-1 : le JAR stocké (`91d61c4591c967e7…`) est identique au JAR testé.

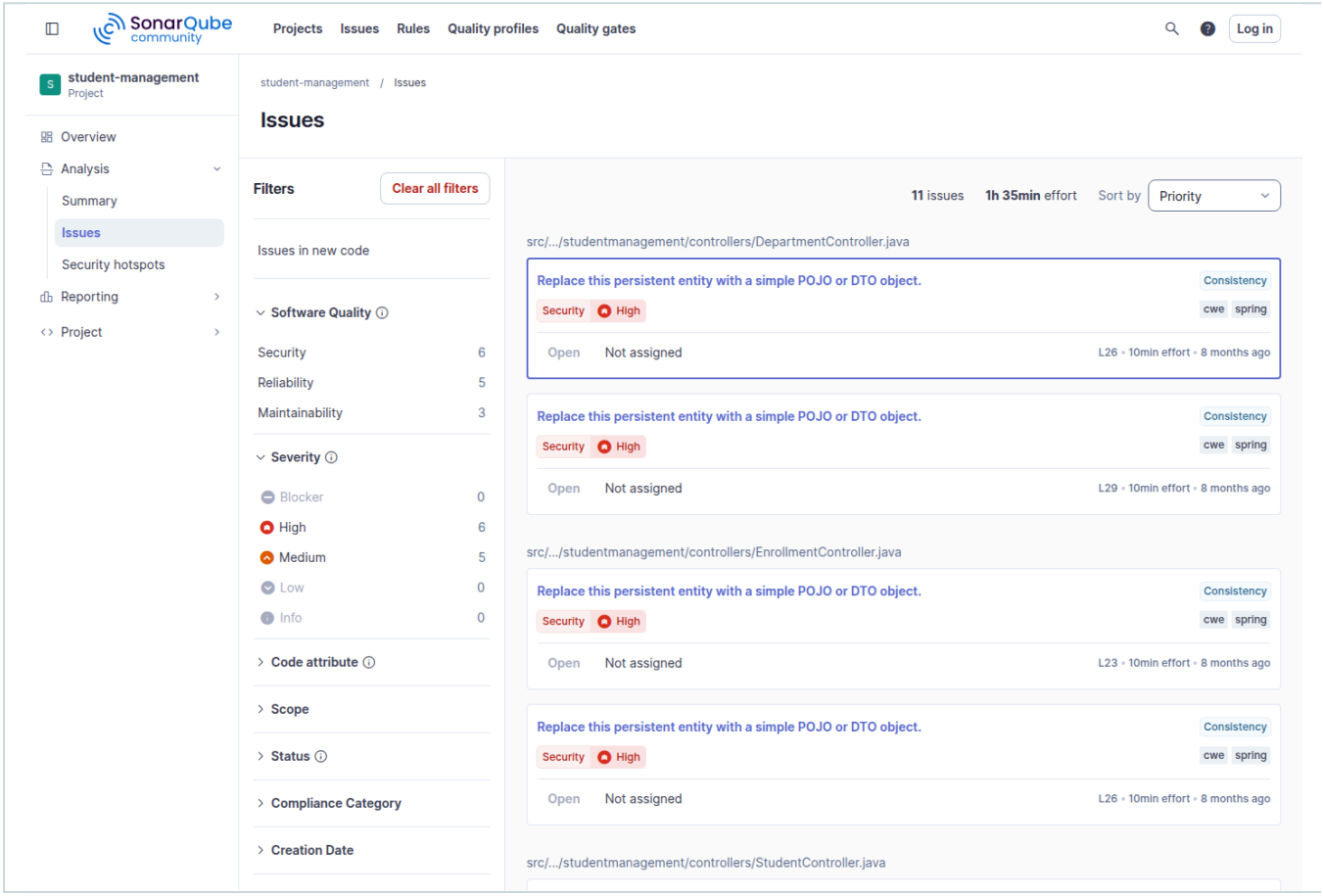


Figure 1. SonarQube : 11 problèmes ouverts, dont 6 de sévérité élevée liés à la liaison directe d'entités JPA dans les contrôleurs.

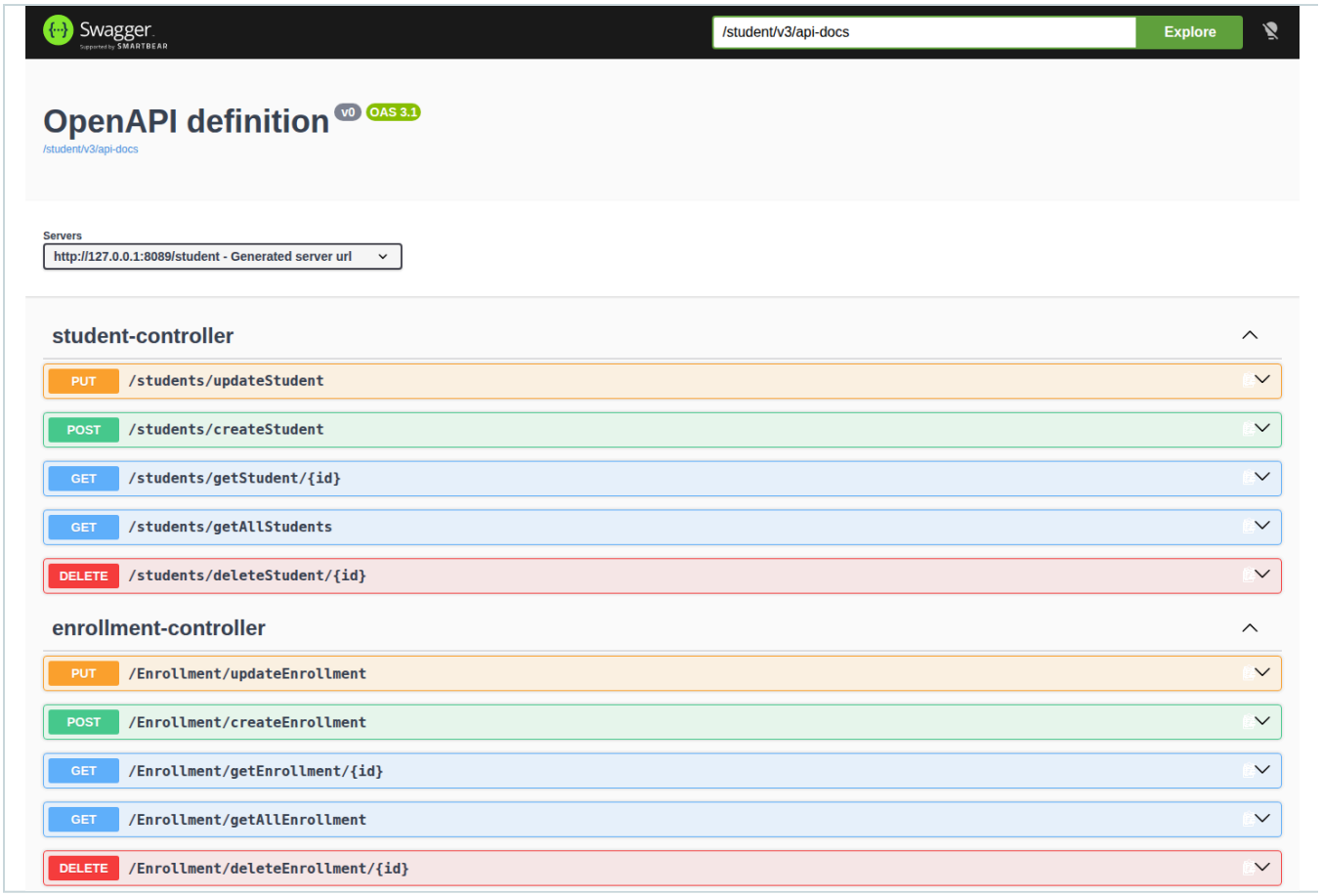


Figure 2. L'API déployée dans le laboratoire éphémère, documentée par Swagger.

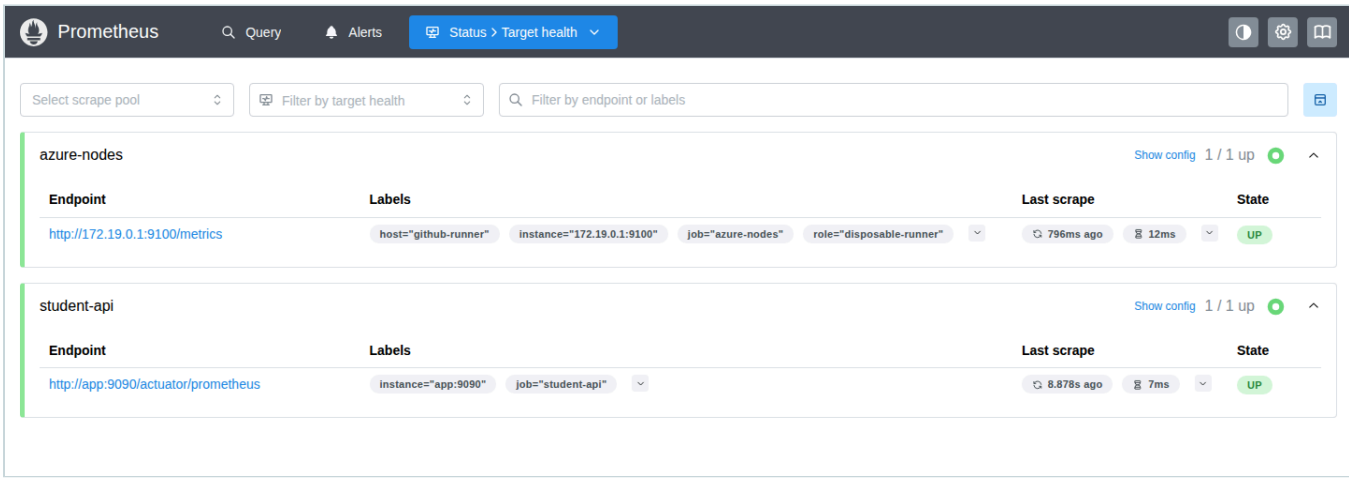


Figure 3. Prometheus : l'API et le Node Exporter du runner sont actifs (le job conserve son nom historique « azure-nodes »).



Figure 4. Grafana : disponibilité, débit, latence et mémoire de l'API pendant l'exécution.

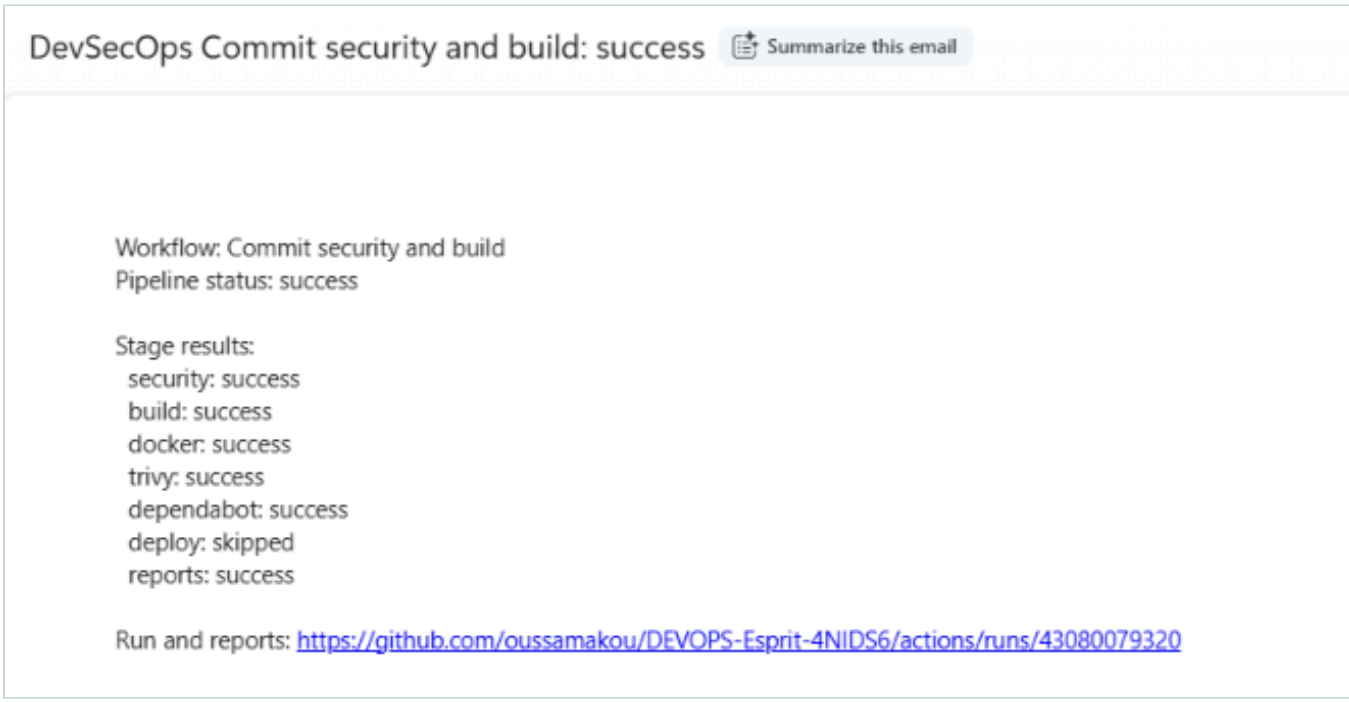


Figure 5. Courriel de statut reçu dans la boîte de réception (exécution 43080079320), adresse masquée.

7. Incidents rencontrés et corrections

Ces incidents réels montrent que les contrôles ont effectivement arrêté la chaîne, et comment chacun a été corrigé.

Date	Constat	Cause	Correction
Octobre 2026	Trivy bloque la première image : 11 vulnérabilités élevées ou critiques.	Versions de Spring Boot, Tomcat et Jackson dépassées.	Passage à Spring Boot 4 et forçage de Jackson 3.1.7 ; nouvelle analyse à 0.
Octobre 2026	Checkov refuse le port HTTP 80 ouvert à Internet (CKV_AZURE_160).	Le port servait à la validation des certificats.	Port fermé, validation TLS-ALPN sur le port 443 ; nouvelle analyse réussie.
23 octobre 2026	Le premier déploiement Compose sur Azure échoue.	Ports non publiés sur des réseaux internes, fichiers de configuration illisibles.	Réseau de pont dédié, fichiers de configuration rendus lisibles (mode 0644) ; déploiement réussi.
Octobre 2026	Nexus signale l'utilisation de sa clé de chiffrement par défaut.	Clé privée non configurée à l'installation.	Clé de 32 octets générée et montée en lecture seule, secrets rechiffrés.
11 novembre 2026	Nmap refuse la première exécution sur le runner GitHub.	Le runner expose son propre service SSH (port 22).	Politique ajustée pour ce seul port ; les ports du laboratoire restent contrôlés.

8. Limites et plan d'action

Limites assumées

- L'environnement est éphémère : aucune donnée ni tableau de bord ne survit à l'exécution.
- Dans ce runner jetable, la consultation anonyme en lecture seule est activée sur SonarQube, Nexus et Grafana pour permettre les captures ; la licence de Nexus Community Edition est acceptée automatiquement à chaque exécution, à la demande du propriétaire du projet.
- La porte de qualité SonarQube porte sur le nouveau code : la franchir ne prouve pas l'absence de problèmes dans le code existant.
- Le balayage Nmap ne couvre que TCP et une liste de ports définie ; UDP n'est pas testé.

Plan d'action

Priorité	Action	Bénéfice attendu
Élevée	Remplacer les entités JPA exposées par des DTO de requête	Supprime les 6 problèmes élevés SonarQube
Élevée	Activer les alertes et mises à jour de sécurité Dependabot	Les vulnérabilités connues seraient signalées automatiquement
Moyenne	Ajouter l'authentification et l'autorisation à l'API	Indispensable avant toute donnée réelle
Moyenne	Traiter les 13 propositions de mise à jour, dont 8 majeures	Réduit la dette de dépendances
Moyenne	Étendre les tests au-delà de 41,5 % de couverture	Mieux protéger contre les régressions
Faible	Rendre la règle de branche master active	Bloque toute fusion sans contrôles réussis

9. Conclusion

La chaîne réalisée applique les principes DevSecOps de bout en bout : les contrôles de sécurité interviennent avant la compilation, un seul artefact traverse toutes les étapes, chaque outil laisse une trace vérifiable et l'équipe est informée à chaque exécution.

La fin de l'abonnement Azure a été transformée en une version sans coût qui conserve toutes les étapes. Les prochaines améliorations ne portent plus sur la chaîne elle-même mais sur ce qu'elle a révélé : corriger les six problèmes de sécurité élevés, activer les alertes Dependabot et protéger l'API.

> Pour le détail de chaque étape, les rapports d'outils originaux et les journaux : rapport complet en anglais, *documentation/FINAL-REPORT.pdf*. Dépôt : https://github.com/oussamakou/DEVOPS-Esprit-4NIDS6. Révision de ce document : `b2f92a3`.